

- ⑤确认应对动力蓄电池系统的状态进行监控。
- ⑥确认应对动力蓄电池系统退出安全状态的条件进行监控。
- ⑦确认结束后，应在确认环境温度下观察 1 个小时。

从这个标准举例中，对危害分析、安全目标、验证方法、确认方法都进行了定义，实际验证与确认过程远比标准所要求的内容多得多，但从采用的方法上讲，这与验证与确认定义是一致的。

当然，验证与确认还包括其他方面，如工具方法、人员能力、过程符合度。实际的工作中，总是利用有限的资源，开发符合系统规定要求的产品。如何利用有限资源，抓住关键，把验证与确认工作做到实处，确保系统的安全要求得以实现，系统的风险得到了控制。这可不是项目中的验证与确认人员能解决的、需要组织中的系统主管、技术经理等高层人员去解决。

17.5.2 嵌入式系统软件的测试

1. 软件测试概述

通常，软件验证的方法主要包括评审（Review）、分析（Analysis）和测试（Testing）等三种方法。而软件测试是软件开发阶段中非常重要的一个过程，是软件验证工作的核心活动。传统的软件测试是基于过程的测试方法。

软件测试存在多种分类方法，以测试过程中程序执行状态为依据可分为静态测试（Static Testing, ST）和动态测试（Dynamic Testing, DT）；以具体实现算法细节和系统内部结构的相关情况为依据可分黑盒测试、白盒测试和灰盒测试三类；从程序执行的方式来分类，可分为人工测试（Manual Testing, MT）和自动化测试（Automatic Testing, AT）。

传统的软件验证与确认活动采用双 V 型的方法，如图 17-21 所示。其中，测试过程主要分为单元测试、集成测试、系统测试和验收测试等 4 个过程。

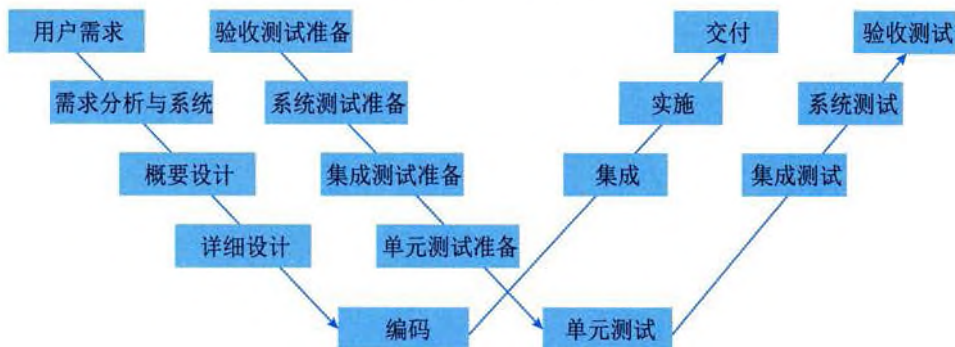


图 17-21 嵌入式软件验证和确认的双 V 型方法

但是，传统测试方法存在以下不可避免的问题：

- (1) 如果测试不够详尽，那么 Bug 就会遗留在产品代码中并潜在地造成严重的问题。
- (2) 测试常常在代码编写完成之后进行。
- (3) 测试人员对软件系统理解不够，会漏掉一些重要内容未被测试到。
- (4) 文档优劣直接影响测试效果。文档和代码往往存在不一致的地方，有可能造成未发现

缺陷。

(5) 如果缺乏自动化测试，不能保证每次重复测试的一致性。

(6) 传统的纠正错误的方法极有可能是在一个地方纠正了错误而又在别的地方引入了新的问题。

2. 基于需求的软件测试方法

民用航空飞行器的机载软件，其安全性是非常重要的，稍有不妥将会引起机毁人亡的重大事故。为了提高机载软件安全性，适航标准中针对机载软件的测试方法制定了详细的标准要求，即 DO-178B/C《机载系统和设备合格审定中的软件考虑》。DO-178B/C 是美国航空无线电技术委员会为支持含有数字计算机的机载系统和设备的研制工作而制定的软件开发准则，是现行的适航标准。DO-178B/C 标准把软件生命周期分为软件计划过程、软件开发过程和软件综合过程，其中，软件开发过程和软件综合过程又分别被细分为 4 个子过程，如表 17-7 所示。

表 17-7 DO-178B/C 中的软件生命周期过程

软件生命周期过程	包含子过程	DO-178B/C 章节
软件计划过程		第四章
软件开发过程	软件需求过程	第五章
	软件设计过程	
	软件编码过程	
	集成过程	
软件综合过程	软件验证过程	第六章
	软件配置管理过程	第七章
	软件质量保证过程	第八章
	审定联络过程	第九章

其中，软件验证过程规定了机载软件的验证与确认活动。它主要包含对软件产品和软件验证结果进行技术评估，以保证其正确性、合理性、完好性、一致性、无歧义性等特性的所有活动。软件验证依然是一项十分复杂的活动，DO-178B/C 中根据验证活动的分类，分别在 DO-178B/C 标准的附件 A 的表 A-3、A-4、A-5、A-6、A-7 列出了这些验证活动应该实现的目标。

DO-178B/C 提出了基于需求的软件测试方法。将软件测试过程分为三种测试方法：软硬件集成测试、软件集成测试、低级需求的测试。软硬件集成测试是运行在目标机上的，软件集成测试和低级需求的测试环境没有要求。基于需求的软件测试过程见图 17-22 所示。

软件测试可能是一个递进、迭代的过程，通过测试用例的开发可覆盖所有的软件需求，通过运行测试用例可覆盖所有软件代码，并且不断分析，如果有遗漏的需求或代码没有被覆盖，则要增加相应的测试，直到所有需求和代码都被覆盖为止。

基于需求的覆盖率分析（TCA）：分析基于需求测试用例对软件需求的覆盖程度。基于结构的覆盖率分析（SCA）：分析基于需求测试用例对代码结构的覆盖程度。主要包括：①结构覆盖率分析在源代码上进行；②确认结构覆盖率已经达到软件级别要求的程度；③对 A 级软件，如果生成的目标码不能直接追溯到源代码，要进行目标码覆盖率分析（OCA）；④覆盖率标准满足

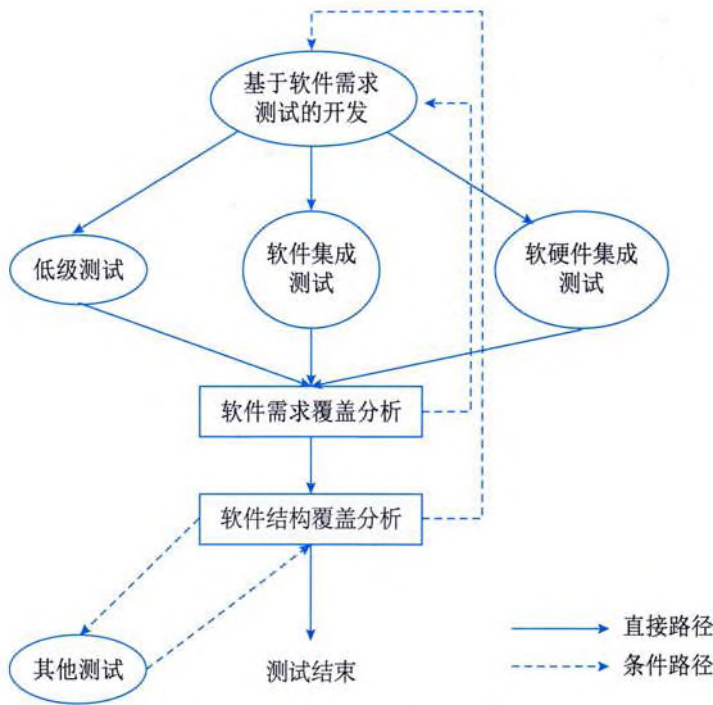


图 17-22 基于需求的软件测试过程

100%TCA 和 SCA。

DO-178B/C 关于不同等级的软件有不同的软件结构覆盖要求，这在 DO-178B/C 表 A-7 验证过程结果的验证中有明确的要求，摘录如表 17-8 所示。

表 17-8 软件级别对应的结构覆盖分析

目 标	软件等级
改进条件 / 判定覆盖 (Modified Condition/Decision Coverage, MCDC)	A
判定覆盖 (Decision Coverage)	A, B
语句覆盖 (Statement Coverage)	A, B, C

语句覆盖是指程序中的每条语句都被调用过至少一次。

判定覆盖是指程序中的每个入口、出口都被调用至少一次，并且程序中每个判定的每个可能结果都至少取到一次。

改进条件 / 判定覆盖是指程序中的每个入口、出口都被调用至少一次，程序中每个条件的每个可能结果都至少被取到一次，程序中每个判定的每个可能结果都至少取到一次。判定中的每个条件都能够被显示地表达，此条件可独立影响判定结果。要让一个条件能独立影响判定结果，可以通过固定其他可能的条件，只更改这个条件来实现。

软件结构覆盖可以在软件源代码或目标码上进行，使用软件覆盖工具进行分析一般需要先对软件代码进行插桩修改，以记录软件运行的路径条件等信息，测试执行结束后将其传输至后台软件进行分析。后台软件分析覆盖数据，得出软件运行的覆盖结果。对这些覆盖结果还要进行人工分析，分析代码结构没有被覆盖的原因。

(1) 如果未覆盖代码有需求而无测试用例，则要增加测试用例。

(2) 如果未覆盖代码无需求，则要增加需求，并设计相应测试用例。注意，增加需求时应该提出问题报告以记录需求的缺失。

(3) 如果是死代码，即因为设计错误而永远无法被执行或使用的代码，则要提出问题报告来删除。

(4) 如果是未激活代码，即不准备被运行或使用的代码，或在特定硬件的配置条件下才会被执行的代码。则要给出分析描述这些代码是正确的，并且这些代码不会被错误地执行到。

根据 A 级软件要求，软件测试要满足 100%MCDC 覆盖的要求。100% 覆盖即所有软件代码都要被测试所运行到，未运行到的代码也用上面所说的方法进行分析，并给出合理的分析结果。

覆盖分析的工作可以在测试程序基本开发运行完成后，即测试的后期才执行。这时运行覆盖可以得到较高的覆盖率，相对节省时间。另外，也可以和一般的测试同步进行，即从开始就在两种代码上运行，这时检查代码的覆盖情况可以看出代码是否按预期运行，这对保证代码的正确执行有好处，当然这种方法花费的人力和时间成本也是巨大的。这两种方法有利有弊，而 DO-178B/C 并没有规定覆盖应该在哪个阶段开始进行，可以根据自己的需要选择，当然最终覆盖结果是要求基于最终版代码的。

3. 形式化方法

形式化方法的本质是基于数学的方法来描述目标软件系统属性的一种技术。不同的形式化方法的数学基础是不同的，有的以集合论和一阶谓词演算为基础（如 Z 和 VDM），有的则以时态逻辑为基础。

形式化方法需要形式化规约说明语言的支持。形式化方法的一个重要研究内容是形式规约（Formal Specification，也称形式规范或形式化描述），它是对程序“做什么”（what to do）的数学描述，是用具有精确语义的形式语言书写的程序功能描述，它是设计和编制程序的出发点，也是验证程序是否正确的依据。

形式化方法的意义在于能帮助发现其他方法不容易发现的系统描述不一致、不明确或不完整的情况，有助于增加软件开发人员对系统的理解，因此形式化方法是提高软件系统，特别是安全攸关系统的安全性、与可靠性的重要手段。

针对不同的系统，需要采用不同的形式化方法，每种形式化方法都有不同的数学定义，可通过类型分析决定应用程序采用何种形式化方法。通常，形式化方法分为如下几类：

(1) 面向对象的形式化方法，通过定义状态和操作进行建模，如 Z 语言、VDM、B、Object-Z 等方法。

(2) 面向属性的形式化方法，如 OBJ3、Larch 等方法。

(3) 基于并发性的形式化方法，如 CCS、ACP、CSP、LOTOS 等方法。

(4) 基于实时性的形式化方法，如 TRIO、RTOZ 等方法。

如果根据描述方式，形式化方法还可划分为：模型描述的形式化方法，即模型描述的形式化方法通过构造一个数学模型来直接描述系统或程序；性质描述的形式化方法，即性质描述的形式化方法通过对目标软件系统中不同性质的描述来间接描述系统或程序。